

Briefly describe the artifact. What is it? When was it created?

The artifact is an Android inventory management application that I created in CS 360 Mobile Architecture and Programming. It was developed as the final project for that course. The app was built in Java using Android Studio and allows users to create accounts, log in, and manage inventory items using create, read, update, and delete functions. The app stores data using a SQLite database and includes an alert feature when an item reaches a quantity of zero.

Justify the inclusion of the artifact in your ePortfolio. Why did you select it? What specific components showcase your skills? How was it improved?

I selected this artifact because it shows several software development skills in one project, including Android development, user interface design, object-oriented programming, and database management. I also selected it because it gave me a strong project to improve instead of starting from scratch.

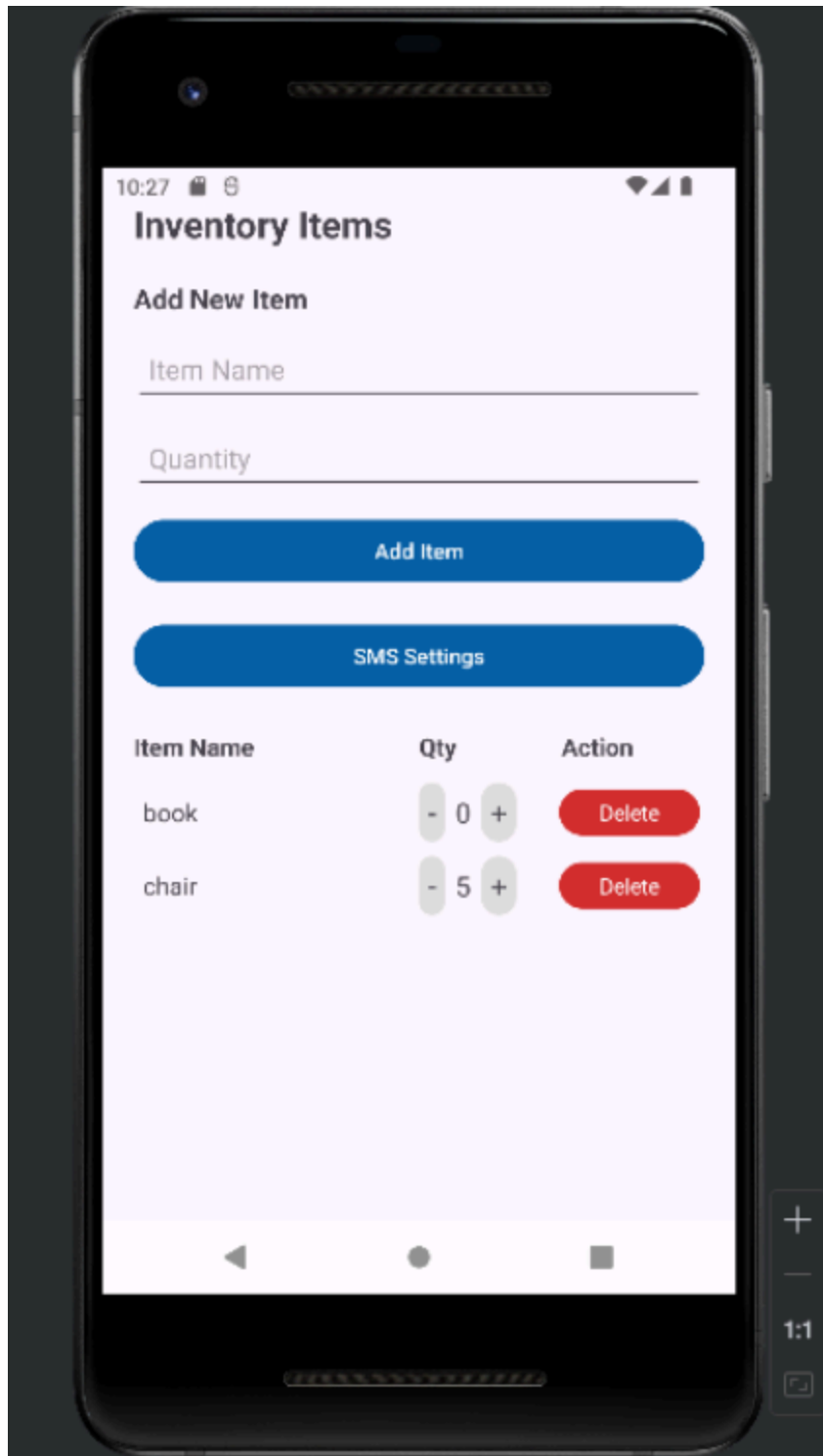
The biggest improvement in this enhancement is the full-screen item detail view. Before, inventory items only showed the name and quantity in a row with no way to see more information or edit the item on its own screen. Now when a user taps an item, a detail screen opens and displays the item name, quantity, category, and price in a more organized layout.

I also added a color-coded stock status banner to improve usability. If the quantity is zero, the banner turns red and shows “Out of Stock.” If the quantity is between one and five, it turns yellow and shows “Low Stock.” If the quantity is six or higher, it turns green and shows “In Stock.” This makes it easier for users to quickly understand item stock levels.

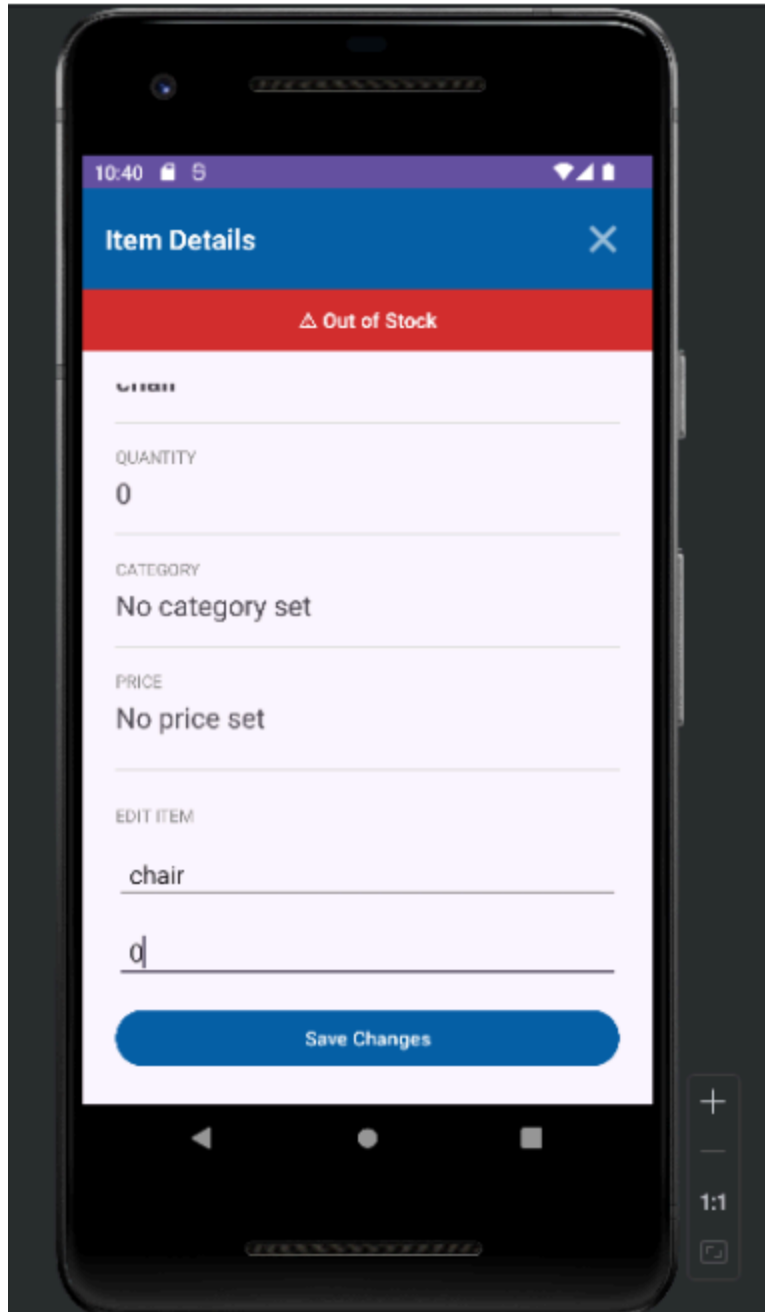
Another improvement is the ability to edit item information directly from the detail screen. Users can update the item name and quantity without returning to the inventory page, and changes update immediately after saving. I also improved input validation by blocking negative quantities and item names longer than fifty characters. This helps prevent bad data from being saved and improves the reliability of the app.

Below are screenshots of the enhanced inventory application that demonstrate some of the improvements made during this milestone. The screenshots show the updated inventory view and the new item detail screen, including the stock status indicators that provide visual feedback based on inventory quantity.

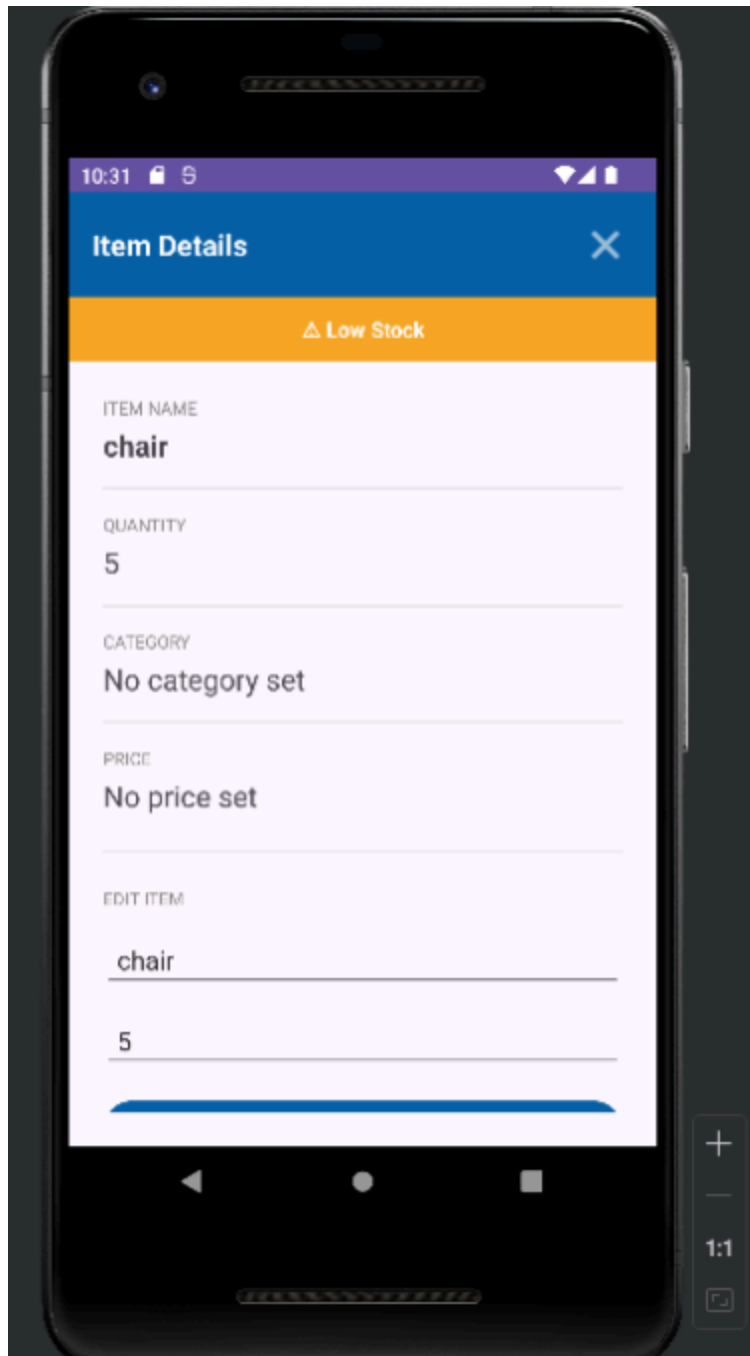
Screenshot 1: Inventory list screen (shows items in the app)



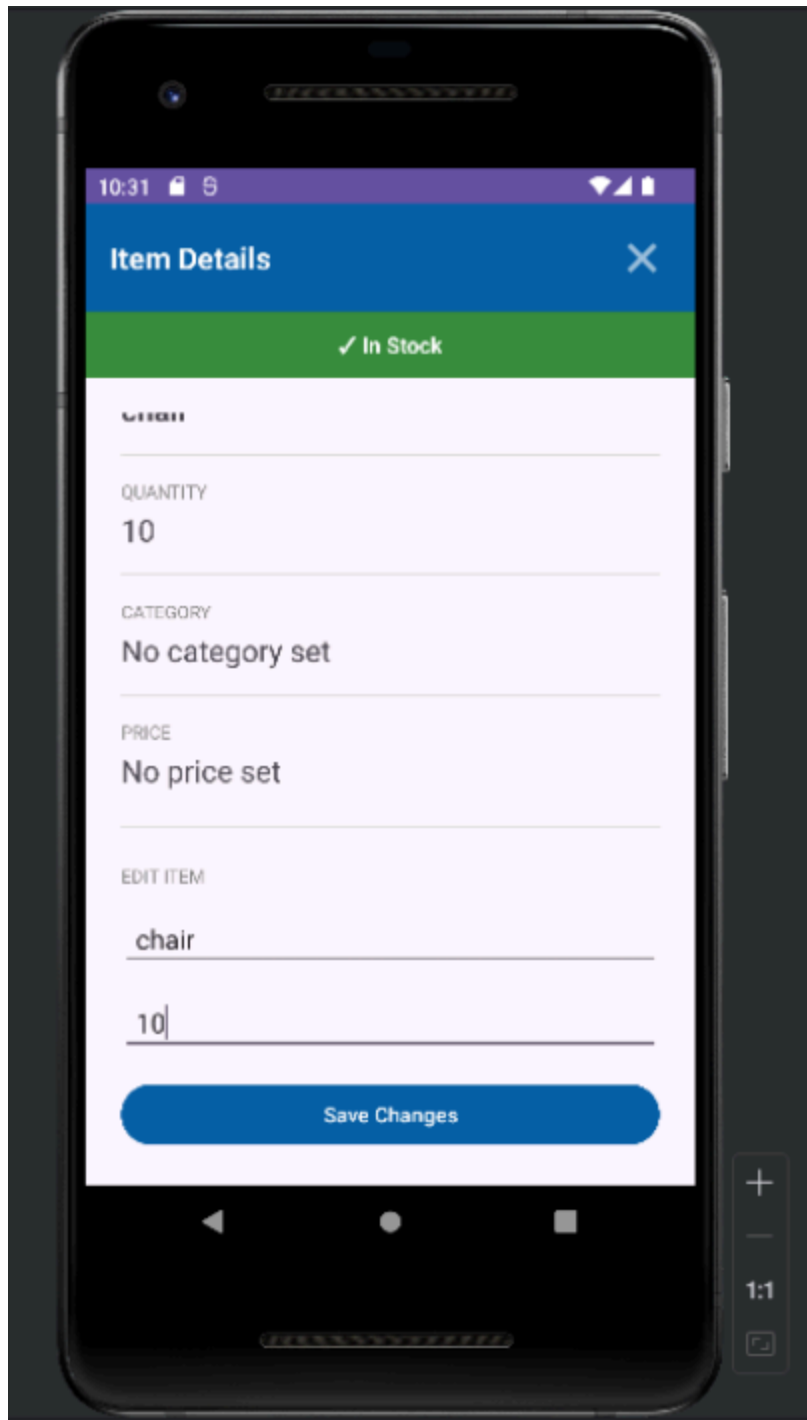
Screenshot 2: Item detail screen with quantity **0** for “Chair” showing the red **Out of Stock** banner



Screenshot 3: Item detail screen with quantity **5** for “Chair” showing the **yellow Low Stock** banner



Screenshot 4: Item detail screen with quantity **10** for “Chair” showing the **green In Stock** banner



Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome coverage plans?

Yes, this enhancement met the course outcomes planned in Module One. Course Outcome 4 was met because the detail view, stock status banner, and editing features demonstrate Android development skills and improve the user experience. Course Outcome 5 was also met because

input validation helps protect the database from invalid data by blocking negative quantities and long item names. As of right now there are no updates to my outcome coverage plans. The remaining outcomes will be addressed in the next two enhancement categories.

Reflect on the process of enhancing and modifying the artifact. What did you learn? What challenges did you face?

One thing I learned from this enhancement was how Android handles navigation between screens and updates data. I learned how to pass information between activities and refresh the inventory list after changes were made. One challenge I faced was that edited items were not updating in the inventory screen after returning from the detail page. I fixed this by using the `onResume()` method so the inventory reloads when the user comes back to the screen.

I also learned the importance of separating features into different classes. Moving the detail screen and editing features into a separate activity made the code cleaner, easier to understand, and easier to manage.